TASK-3 SHORT PDF

Design and Simulation of D Flip-Flop, JK Flip-Flop, 4-Bit Register and Counter Using Verilog HDL

---

1. D FLIP-FLOP

Theory

A D (Data) Flip-Flop is a sequential circuit used for storing one bit of information. It has one data input (D), one clock input (CLK), and one output (Q). The output changes only on the active edge of the clock signal.

The characteristic equation of a D Flip-Flop is:

Q(next) = D

Thus, the output follows the input at every positive edge of the clock.

Block Diagram

Inputs:

- D (Data Input)
- CLK (Clock)

Output:

- Q

Verilog Design Logic

The D Flip-Flop is implemented using an always block that triggers on the positive edge of the clock.

```
always @(posedge CLK)

begin

  Q <= D;

end
```

Simulation Results

Different values of D were applied while the clock was running. The output Q changed only at the rising edge of the clock and retained its value between clock edges.

```
/d/verilogcodes

HP@LAPTOP-1I469TMH MINGW64 ~
$ cd /d/verilogcodes

HP@LAPTOP-1I469TMH MINGW64 /d/verilogcodes
$ iverilog -o d_flipflop.vvp d_flipflop.v

HP@LAPTOP-1I469TMH MINGW64 /d/verilogcodes
$ iverilog -o d_flipflop_tb.vcd d_flipflop_tb.v d_flipflop.v

HP@LAPTOP-1I469TMH MINGW64 /d/verilogcodes
$ vvp d_flipflop_tb.vcd
VCD info: dumpfile d_flipflop.vcd opened for output.
Time    CLK     D       Q
0       0       0       x
5000    1       0       0
10000   0       1       0
15000   1       1       1
20000   0       0       1
25000   1       0       0
30000   0       1       0
35000   1       1       1
40000   0       1       1
45000   1       1       1
d_flipflop_tb.v:36: $finish called at 50000 (1ps)
50000   0       1       1
```

Observation

The simulation confirmed that the output Q follows the value of D only at the positive edge of the clock. This verifies the correct operation of the D Flip-Flop as a one-bit storage element.

---

2. JK FLIP-FLOP

Theory

The JK Flip-Flop is a universal flip-flop that overcomes the invalid state of the SR Flip-Flop. It has two inputs, J and K, along with a clock signal.

Truth Table

J	K	Q(next)	Operation
0	0	Q	No Change
0	1	0	Reset
1	0	1	Set
1	1	$\bar{Q}$	Toggle

Verilog Design Logic

The JK Flip-Flop was implemented using a case statement inside a positive edge triggered always block.

always @(posedge CLK)

begin

```
    case({J,K})
        2'b00: Q <= Q;
        2'b01: Q <= 1'b0;
        2'b10: Q <= 1'b1;
        2'b11: Q <= ~Q;
    endcase
end
```

Simulation Results

Various combinations of J and K were applied.

J	K	Operation
0	0	Hold
0	1	Reset
1	0	Set
1	1	Toggle

```
/d/verilogcodes

HP@LAPTOP-1I469TMH MINGW64 /d/verilogcodes
$ vvp jk_flipflop_tb.vcd
jk_flipflop_tb.vcd: Unable to open input file.

HP@LAPTOP-1I469TMH MINGW64 /d/verilogcodes
$ iverilog -o jk_flipflop_tb.vcd jk_flipflop_tb.v jk_flipflop.v

HP@LAPTOP-1I469TMH MINGW64 /d/verilogcodes
$ vvp jk_flipflop_tb.vcd
VCD info: dumpfile jk_flipflop.vcd opened for output.
Time    J       K       CLK     Q
0       0       0       0       x
5000    0       0       1       x
10000   0       1       0       x
15000   0       1       1       0
20000   1       0       0       0
25000   1       0       1       1
30000   1       1       0       1
35000   1       1       1       0
40000   1       1       0       0
45000   1       1       1       1
jk_flipflop_tb.v:46: $finish called at 50000 (1ps)
50000   1       1       0       1

HP@LAPTOP-1I469TMH MINGW64 /d/verilogcodes
$ |
```

The waveform showed that output Q changed only on the rising edge of the clock.

Observation

The JK Flip-Flop successfully performed hold, set, reset, and toggle operations according to the applied input combinations. The simulation matched the theoretical truth table.

3. 4-BIT REGISTER

Theory

A register is a group of flip-flops used for storing binary information. A 4-bit register stores four bits of data simultaneously.

Inputs:

- Clock (CLK)
- Data Input D[3:0]

Output:

- Stored Data Q[3:0]

The register captures and stores the input data at the positive edge of the clock.

Verilog Design Logic

```
always @(posedge CLK)
begin
  Q <= D;
end
```

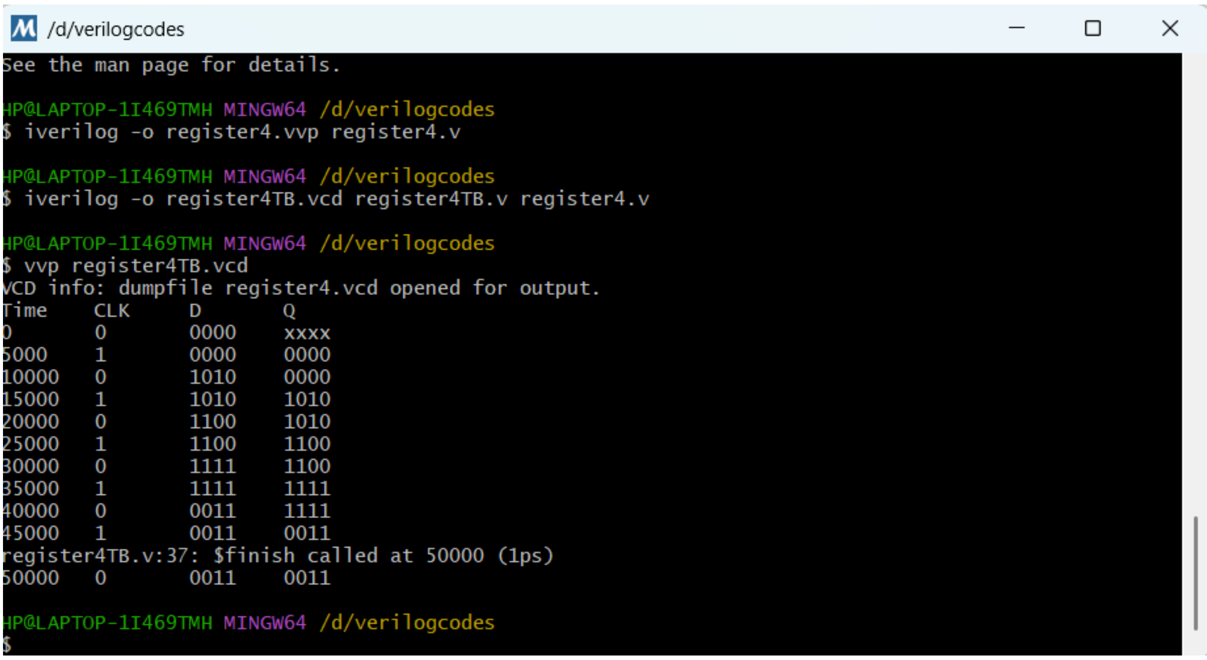
The entire 4-bit input is transferred to the output whenever a rising clock edge occurs.

Simulation Results

The following input values were applied:

D Input	Q Output
0000	0000
1010	1010
1100	1100
1111	1111
0011	0011

The output changed only at the clock transitions and stored the applied input values.



Observation

The register correctly stored and retained the 4-bit data values. The output remained stable between clock edges and updated only when triggered by the clock signal.

---

4. MOD-4 COUNTER

Theory

A counter is a sequential circuit that progresses through a predefined sequence of states in response to clock pulses.

The implemented counter is a MOD-4 counter which counts from 0 to 3 and then repeats.

Counting Sequence

Decimal	Binary
0	00
1	01
2	10
3	11
0	00

Verilog Design Logic

always @(posedge CLK)

```
begin
  if(Q == 2'b11)
     Q <= 2'b00;
  else
     Q <= Q + 1;
end
```

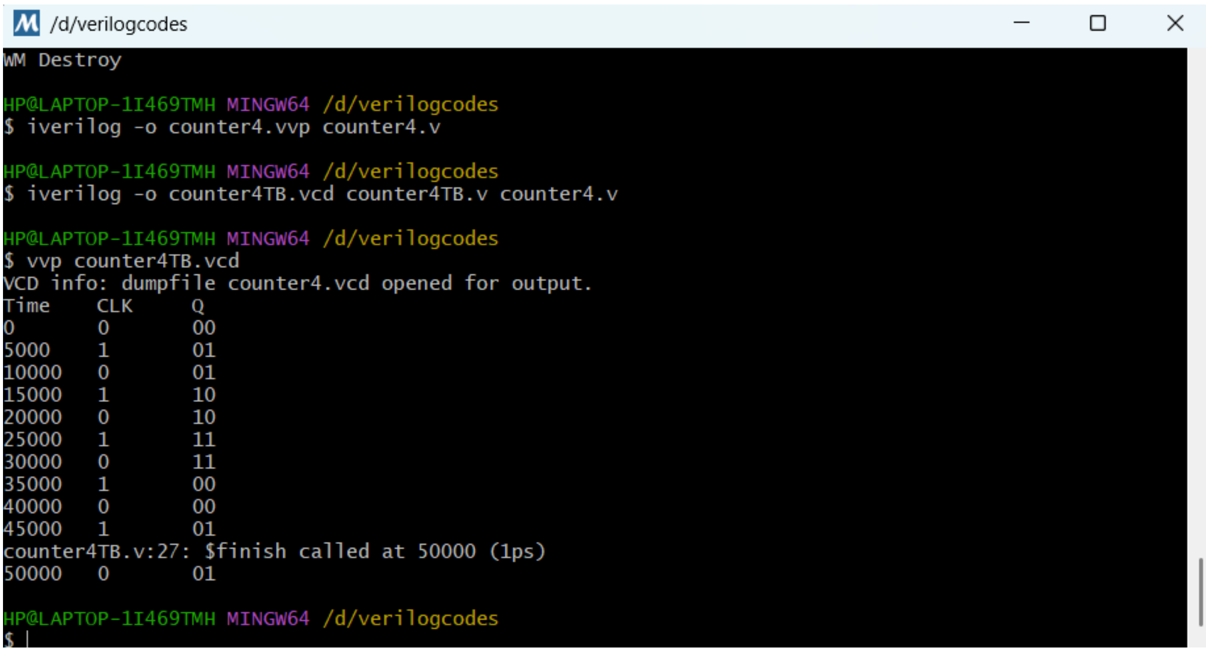
The counter increments at every positive edge of the clock and resets to 00 after reaching 11.

Simulation Results

Observed count sequence:

Clock Pulse	Output
1	00
2	01
3	10
4	11
5	00
6	01

The waveform displayed continuous counting in cyclic order.


```
/d/verilogcodes
WM Destroy

HP@LAPTOP-1I469TMH MINGW64 /d/verilogcodes
$ iverilog -o counter4.vvp counter4.v

HP@LAPTOP-1I469TMH MINGW64 /d/verilogcodes
$ iverilog -o counter4TB.vcd counter4TB.v counter4.v

HP@LAPTOP-1I469TMH MINGW64 /d/verilogcodes
$ vvp counter4TB.vcd
VCD info: dumpfile counter4.vcd opened for output.
Time    CLK     Q
0       0       00
5000    1       01
10000   0       01
15000   1       10
20000   0       10
25000   1       11
30000   0       11
35000   1       00
40000   0       00
45000   1       01
counter4TB.v:27: $finish called at 50000 (1ps)
50000   0       01

HP@LAPTOP-1I469TMH MINGW64 /d/verilogcodes
$ |
```

Observation

The counter successfully counted through all four states and restarted from zero after reaching the maximum count. The simulation verified proper modulo-4 operation.

Overall Results

The Verilog HDL models for D Flip-Flop, JK Flip-Flop, 4-Bit Register, and MOD-4 Counter were successfully designed and simulated. The outputs obtained through Icarus Verilog and GTKWave matched the expected theoretical behavior of each sequential circuit.

Conclusion

The experiment demonstrated the design and simulation of fundamental sequential logic circuits using Verilog HDL. The D Flip-Flop stored one bit of information, the JK Flip-Flop performed hold, set, reset, and toggle operations, the 4-Bit Register stored multi-bit data, and the MOD-4 Counter generated a cyclic counting sequence. The simulation results validated the correctness of the designs and provided practical understanding of sequential circuit operation.